



Primer parcial de práctica 27/05

TEMA 1: Gestión de Configuración y Versionado

Este documento introduce los fundamentos de la **Gestión de Configuración**, centrándose específicamente en el **Versionado** como una práctica esencial para el desarrollo profesional.

1. Definiciones Fundamentales

- **Sistema:** Se define como la combinación de elementos organizados (hardware, código, documentación, requerimientos, pruebas, etc.) para cumplir objetivos específicos.
- **Gestión de Configuración:** Es la aplicación de prácticas para identificar y documentar los ítems configurables de un producto, controlando y registrando todo el ciclo de vida de los cambios (aprobación, implementación, verificación).

2. Elementos de la Gestión de Configuración

Dentro de esta disciplina, encontramos tres pilares principales:

1. **Sistemas de control de versiones:** No se limitan solo al código fuente.
2. **Herramientas de integración / construcción:** Enfocadas en la automatización (CI/CD).
3. **Sistemas de control de cambios:** Gestión de incidencias o "issues".

3. ¿Por qué versionar? (Pros y Contras)

El uso de estas herramientas ofrece una estructura sólida para el trabajo en equipo:

Ventajas (+)	Desventajas / Desafíos (-)
Acceso de todo el equipo a las versiones del código.	Requiere disciplina personal y grupal.
Capacidad de revertir cambios y aislar errores.	Implica seguir un flujo de trabajo estricto.

Ventajas (+)	Desventajas / Desafíos (-)
Facilita la auditoría y el <i>code review</i> .	Requiere capacitación previa.
Base para la automatización (CI/CD).	Puede haber arranques "en frío".
Visibilidad, trazabilidad y backup constante.	-

4. Herramientas y Tecnologías

- **Tecnologías de control:** Git (la más utilizada actualmente), Subversion, Mercurial.
- **Plataformas de hosting:** GitHub, GitLab, Bitbucket.

5. Conceptos Clave de Git

- **Repositorio:** El proyecto completo, una copia de todo el código.
- **Ramas (Branches):** Separaciones del código para diferentes contextos (features, fixes, master, QA).
- **Etiquetas (Tags):** Marcas asociadas a commits específicos para identificar hitos o versiones (v1.0, v2.0).

6. Flujo de Trabajo (Morning Routine)

El proceso diario típico de un desarrollador sigue estos pasos:

1. `git pull` : Descargar o actualizar la copia local con los cambios del servidor remoto.
2. **Trabajo local:** Generar nuevos archivos o modificar los existentes.
3. `git add` : Pasar los cambios al área de preparación (*stage*).
4. `git commit` : Registrar los cambios localmente.
5. `git push` : Enviar los cambios locales al servidor remoto.

Nota sobre Conflictos: Ocurren cuando dos desarrolladores modifican el mismo archivo simultáneamente. Para resolverlo, es necesario sincronizar la copia local, unificar versiones con el par y actualizar el servidor remoto.

Posibles Preguntas de Examen

1. ¿Qué es la Gestión de Configuración y qué relación tiene con el versionado?

- *Respuesta sugerida:* Es la práctica de identificar, documentar y controlar los cambios en los ítems de un producto. El versionado es una de las herramientas dentro de esta gestión para controlar las evoluciones del código y otros elementos.

2. Explique la diferencia entre una Rama (Branch) y una Etiqueta (Tag).

- *Respuesta sugerida:* Las ramas son separaciones del código para trabajar en diferentes contextos (como una funcionalidad nueva), mientras que las etiquetas son marcas estáticas en un commit específico para identificar versiones o hitos.

3. Describa el escenario de un conflicto de versiones y cómo se resuelve.

- *Respuesta sugerida:* Ocurre cuando el "Dev A" intenta subir cambios pero el "Dev B" ya actualizó los mismos archivos en el servidor. Se resuelve sincronizando (`pull`), unificando versiones manualmente con el otro desarrollador y luego subiendo la versión final (`push`).

4. ¿Cuáles son los pasos iniciales (Paso 0) para empezar un proyecto con Git?

- *Respuesta sugerida:* 1. `git init` (crear repo), 2. `git commit` (inicializar), 3. Crear copia remota (GitHub), 4. `git push` (sincronizar).
-
-

TEMA 2: Requerimientos y Spec-Driven Development (SDD)

Este módulo aborda la transición desde los métodos tradicionales y ágiles hacia nuevas formas de especificación necesarias para trabajar eficientemente con Inteligencia Artificial.

1. Conceptos de Requerimientos y Discovery

- **Definición:** Un requerimiento es una propiedad o funcionalidad que el producto debe

tener para resolver un problema real.

- **Clasificación:** Se dividen en funcionales/no funcionales (atributos de calidad) y requerimientos del producto/proyecto.
- **Discovery:** Proceso inicial y continuo para comprender el problema mediante la interacción con los *stakeholders*.
- **Técnicas de Elicitación:** Incluyen entrevistas, escenarios (casos de uso), prototipos (mockups) e historias de usuario.

2. Historias de Usuario e IA

- **Formato tradicional:** "Como `<rol>` quiero `<funcionalidad>` para `<beneficio>`".
- **Elementos adjuntos:** Deben incluir valor de negocio, estimación y condiciones de aceptación.
- **El rol de la IA:** A diferencia de los humanos, la IA no puede tolerar la incertidumbre. Los **criterios de aceptación** son fundamentales para limitar al modelo y evitar "alucinaciones".

3. El auge del Spec-Driven Development (SDD)

Frente al **Vibe Coding** (programar solo con prompts conversacionales, lo cual es inestable y difícil de versionar), surge el **SDD** como la alternativa profesional.

Característica	Vibe Coding (Amateur)	SDD (Profesional)
Punto de inicio	Prompt conversacional volátil.	Archivo Markdown (.md) estructurado.
Entorno	Chat o terminal aislada.	Repositorio Git (Versionable).
Dirección Técnica	La IA adivina arquitectura y patrones.	Límites explícitos y tecnologías forzadas.
Flujo de Entrega	Riesgo de código roto entregado de golpe.	Iterativo, validado tarea por tarea.

4. Anatomía de una Especificación (Spec)

Un documento de spec debe ser en texto plano (Markdown), entendible tanto por humanos

como por agentes de IA, especificando el **qué** pero dejando el **cómo** (algoritmos) al agente.

Partes de una Spec:

1. **Contexto:** Justificación de la funcionalidad para el agente.
2. **Requisitos:** Especificidad extrema sobre lo que se debe construir.
3. **Restricciones:** Tecnologías innegociables (ej: usar PostgreSQL vía Prisma).
4. **Fuera de Alcance:** Qué NO debe hacer la IA para prevenir alucinaciones.
5. **Métricas de Éxito:** Criterios de aceptación para tests automáticos.

5. Pasos para Implementar SDD

1. **Alineación:** Solicitar que el agente pregunte dudas antes de codificar e incorporar referencias visuales (figma, capturas).
 2. **Marcar Límites:** Definir restricciones tecnológicas, de datos y funcionales (ej: tamaño máximo de archivos).
 3. **Subtareas Ordenadas:** Dividir la ejecución en tareas específicas para agentes especializados (DB, APIs, UI/UX) y validar cada commit.
-

Posibles Preguntas de Examen

1. **¿Por qué el "Vibe Coding" se considera insostenible a nivel profesional?**
 - *Respuesta sugerida:* Porque genera anarquía técnica (la IA decide patrones y frameworks), no es fácilmente versionable y presenta alta incertidumbre y ambigüedad en los resultados finales.
2. **¿Cuál es la función principal de los criterios de aceptación al trabajar con IA?**
 - *Respuesta sugerida:* Actúan como límites para que la IA no alucine y son la base para la generación automática de casos de prueba que verifiquen si lo implementado es correcto.
3. **Explique la diferencia entre el enfoque de Waterfall, Agile e IA según la evolución del desarrollo.**
 - *Respuesta sugerida:* Waterfall planifica a largo plazo entregando tarde; Agile entrega algo usable en cada paso; la IA entrega el producto rápido pero requiere "shaping" (dar forma),

propiedad y verificación constante por parte del desarrollador.

4. ¿Qué debe contener la sección de "Fuera de Alcance" en una spec y para qué sirve?

- *Respuesta sugerida:* Debe contener acciones específicas que el sistema no debe realizar (ej: no añadir dependencias externas sin permiso). Sirve principalmente para prevenir alucinaciones y mantener el foco del agente de IA.
-
-

TEMA 3: Gestión de Incidencias y Trazabilidad

Este módulo profundiza en la gestión de incidencias como un pilar de la Gestión de Configuración, conectando los requerimientos con la implementación y el mantenimiento del software.

1. Conceptos Fundamentales

- **Gestión de Incidencias:** Los sistemas de seguimiento brindan información sobre el tipo de incidencia, su evolución, documentos adjuntos, tiempos de resolución, autores y código fuente involucrado.
- **Trazabilidad:** Es el registro del vínculo entre requerimientos, funcionalidades, pruebas y código.
- **Propósito:** Minimizar el riesgo de errores y mantener un registro claro de la evolución de cada elemento del sistema.
- **Integración:** Resulta imperativo integrar los sistemas de seguimiento de incidencias con los sistemas de versionado (como Git) para una gestión integral.

2. Flujos de Trabajo (Workflows)

Las incidencias atraviesan diferentes estados según la herramienta y el proceso definido. Algunos ejemplos comunes son:

- **Básico:** Open > In progress > Close.
- **Extendido:** Nuevo > En progreso > Resuelto > Pendiente de aprobación > Cerrado.

- **Por fases:** Nuevo > Análisis > Diseño > Desarrollo > Pruebas > Finalizado.

3. El Cambio de Paradigma: Del Agile Tradicional a la IA

El enfoque ágil tradicional (Product Backlog, Sprint Planning) buscaba gestionar la incertidumbre mediante ceremonias. Con la IA, el flujo evoluciona hacia el **HITL (Human In The Loop)**.

Fases del flujo HITL:

1. Prototipado.
2. SDD (Spec Driven Development) con límites.
3. Plan sobre tareas.
4. Implementación.
5. Validación y verificación.

El Cono de Incertidumbre

La precisión de las estimaciones varía según la fase del proyecto. Al inicio, la incertidumbre es máxima (hasta 4x), y se reduce a medida que se completan los requerimientos, el diseño y los prototipos.

4. Los "3 Carriles" de Velocidad

En 2026, el desarrollo se divide en tres velocidades concurrentes:

- **Carril Interno (Inner):** Prototipado a velocidad de IA (vibe-coding) realizado por PMs o perfiles no técnicos para validar ideas rápidamente.
- **Carril Medio (Middle):** Conversión de ideas a productos aptos para producción usando IA supervisada y enfoques multi-agente.
- **Carril Externo (Outer):** Velocidad controlada por humanos para lo que se pasa a producción, basado en feedback de usuarios y visión a largo plazo.

5. Evolución de los Roles: El Surgimiento del "Builder"

Las fronteras entre roles se han difuminado, eliminando la "cadena de montaje" tradicional donde el PM escribía, el UX diseñaba y el Dev programaba.

- **Product Managers:** Ya no crean "documentos muertos"; usan IA y herramientas low-code para prototipos funcionales.
- **UX Designers:** Lanzan interfaces operativas para pruebas reales en lugar de solo pantallas estáticas.
- **Desarrolladores:** Se elevan hacia roles de arquitectura de sistemas y ejecutan validaciones de QA automáticas.

6. Enfoque Híbrido Corporativo

Las empresas aplican una distribución de carga de trabajo específica para mitigar riesgos:

- **Delegación (70%):** Uso intensivo de IA para código repetitivo y pruebas.
- **Intervención Humana (30%):** Mandato estricto para rutas críticas y lógica central.
- **Revisiones Obligatorias:** Auditoría de código asistida por agentes secundarios independientes.

7. Herramientas Comunes

- Redmine.
 - Jira.
 - GitHub / GitLab / Bitbucket Issues.
-

Posibles Preguntas de Examen

1. **¿Por qué es fundamental la trazabilidad en la Gestión de Configuración?**
 - *Respuesta sugerida:* Porque permite vincular requerimientos, funcionalidades, pruebas y código, lo que minimiza errores y permite tener un registro claro de la evolución del producto y el proceso.
2. **Explique las diferencias entre los tres carriles de velocidad propuestos para el desarrollo actual.**
 - *Respuesta sugerida:* El carril interno es para prototipado rápido con IA; el medio es para convertir esas ideas en productos supervisados por humanos y multi-agentes; el externo es el control final humano y feedback de largo plazo.

3. ¿Qué significa el concepto HITL en el contexto de integración de IA?

- *Respuesta sugerida:* Significa "Human In The Loop" (Humano en el ciclo), implicando que el humano interviene en cada fase (prototipado, implementación, validación) para limitar y verificar los resultados de la IA.

4. ¿Cómo ha cambiado el rol del Product Manager según el esquema del "Builder" de 2026?

- *Respuesta sugerida:* Ha pasado de escribir documentos estáticos (PRDs) a utilizar IA y herramientas low-code/no-code para crear y validar prototipos funcionales directamente.

Aquí tienes el resumen del cuarto archivo, enfocado en el **Análisis y Gestión de Riesgos**, completando así la serie de materiales para tu parcial.

TEMA 4: Análisis y Gestión de Riesgos

Este módulo presenta la gestión de riesgos no solo como una tarea administrativa, sino como un pilar fundamental de la ingeniería de software moderna, integrando estándares de seguridad y el uso de IA.

1. Conceptos Fundamentales

- **Definición de Riesgo:** Es la probabilidad de que ocurra una situación que comprometa la integridad de los activos (materiales o humanos), afectando la continuidad de las operaciones.
- **Características del Riesgo:**
 - **Probabilidad:** Qué tan posible es el evento.
 - **Impacto:** Gravedad del efecto si ocurre .
 - **Duración:** Tiempo que dura el efecto.
 - **Criticidad:** Nivel de importancia para el logro de los objetivos.
- **Objetivo de la Gestión:** Controlar los riesgos de forma proactiva para minimizar su impacto, tomar mejores decisiones y aumentar la probabilidad de éxito del proyecto .

2. Proceso de Gestión (Ciclo Continuo)

La gestión es un proceso iterativo que acompaña todo el ciclo de vida del proyecto:

1. **Identificación:** Detectar amenazas que afecten los objetivos.
2. **Análisis y Priorización:** Evaluar probabilidad e impacto para definir prioridades.
3. **Comunicación:** Informar a los involucrados para generar conciencia y tomar decisiones.
4. **Estrategias de Control:** Definir acciones de mitigación y monitoreo continuo.
5. **Resolución:** Implementar acciones y cerrar los riesgos cuando sea posible.

3. Metodologías Comparadas

Característica	Framework SRM (SEI)	MAGERIT (España)
Enfoque	Flexible y adaptable a procesos software.	Estructurado y normativo (Sector Público).
Experiencia	Requiere analistas con más experiencia.	No requiere tanta experiencia previa del auditor.
Fases SRM	Identificar, Analizar, Planificar, Monitorear y Controlar.	Planificación, Análisis de Riesgos y Gestión de Riesgos.

4. El Paradigma de la Seguridad Integrada (2026)

La seguridad ya no es un "parche" final, sino un requerimiento intrínseco que se gestiona mediante una "trinidad" de disciplinas :

- **Arquitectura SPEC:** Diseñar con contratos inquebrantables (SDD).
- **Gestión de Riesgos (SEI):** Descubrir vulnerabilidades proactivamente.
- **Defensas OWASP:** Construir siguiendo estándares tácticos (C1-C10) .

5. Riesgos en el Desarrollo Agéntico (IA)

La delegación de autonomía a agentes de IA expande la superficie de ataque, surgiendo nuevas amenazas :

- **Inyección de Prompts:** Manipulación externa de los objetivos del agente.

- **Exfiltración de Datos:** Fuga de información sensible vía herramientas de terceros.
- **El Problema del "Confused Deputy":** Cuando un agente es secuestrado y usa sus privilegios para ejecutar órdenes hostiles bajo una identidad confiable .

6. Evolución del Rol: El Arquitecto de Especificaciones

En el flujo SDD, la especificación evoluciona de ser un documento pasivo a una **Constitución Ejecutable**. Sus cuatro dimensiones son :

1. **Specification:** Única fuente de verdad (el contrato).
 2. **Security:** Restricciones constitucionales inviolables por construcción .
 3. **Performance:** Límites de recursos para evitar agencia excesiva.
 4. **Compliance:** Verificación continua contra marcos normativos y legales .
-

Posibles Preguntas de Examen

1. **¿Cuál es la diferencia principal entre el enfoque de seguridad clásico y el paradigma de Seguridad Integrada?**
 - *Respuesta sugerida:* El enfoque clásico ve la seguridad como una externalidad o un parche post-mortem. La Seguridad Integrada la considera un requerimiento intrínseco que nace desde la arquitectura (SPEC) y el descubrimiento de riesgos (SEI) antes de la codificación .
2. **Explique el concepto de "Constitución Ejecutable" en el contexto de SDD.**
 - *Respuesta sugerida:* Es una evolución de la especificación técnica que deja de ser texto libre para convertirse en un archivo *machine-readable* que impone restricciones por diseño, prohibiendo explícitamente patrones inseguros durante la generación de código por IA.
3. **¿Qué es el principio de "Least Agency" (Mínima Agencia) en aplicaciones agénticas?**
 - *Respuesta sugerida:* Consiste en otorgar al agente de IA exclusivamente la autonomía, las herramientas y el tiempo mínimo estrictamente necesario para cumplir con la

especificación, reduciendo así la superficie de ataque.

4. **Compare brevemente el Framework SRM del SEI con MAGERIT.**

- *Respuesta sugerida:* SRM es más flexible y específico para software pero requiere más experiencia del analista, mientras que MAGERIT es más estructurado y normativo, diseñado para el sector público y es más accesible para analistas con menos experiencia.

TEMA 5: Entornos de Ejecución y Containerización

Este módulo explora la evolución de los entornos donde corre el software, desde el despliegue tradicional hasta el uso de contenedores para garantizar la paridad entre desarrollo y producción.

1. El Problema de la Paridad de Entornos

- **"En mi máquina funciona":** Es el problema clásico donde el código corre bien localmente pero falla en el servidor debido a diferencias en versiones de lenguajes, librerías o sistemas operativos.
- **Gestión de Configuración:** Para minimizar errores, es vital registrar y versionar no solo el código, sino también la configuración del hardware, librerías y dependencias.

2. Evolución de los Entornos de Ejecución

Existen tres enfoques principales para desplegar aplicaciones:

Enfoque	Descripción	Desafío
Tradicional	Instalación manual de dependencias directamente sobre el Sistema Operativo del servidor.	Conflictos entre aplicaciones y dificultad para escalar.
Virtualización	Uso de Máquinas Virtuales (VM) que	Consumo elevado de

Enfoque	Descripción	Desafío
	emulan hardware completo, incluyendo un SO invitado.	recursos (CPU, RAM) por cada instancia del SO.
Containerización	Paquetes ligeros que comparten el núcleo del SO pero aíslan la aplicación y sus dependencias.	Requiere una curva de aprendizaje inicial para la orquestación.

3. Docker: La Herramienta Estándar

Docker permite "empaquetar" una aplicación con todo lo que necesita para correr, asegurando que se comporte igual en cualquier lugar.

Conceptos Clave de Docker:

- **Docker Hub:** Repositorio central (nube) donde se comparten imágenes públicas y privadas.
- **Imagen:** Es una plantilla de "solo lectura" que contiene el sistema operativo base, librerías y el código de la app.
- **Contenedor:** Es la instancia ejecutable de una imagen. Es ligero, rápido y aislado del resto del sistema.
- **Docker Desktop:** Herramienta visual para gestionar contenedores localmente.

4. Ciclo de Trabajo con Docker

1. **Build:** Se crea una imagen a partir de un archivo de configuración (`Dockerfile`).
2. **Push:** Se sube la imagen al Docker Hub o registro privado.
3. **Pull:** El servidor de producción descarga la imagen exacta.
4. **Run:** Se inicia el contenedor, garantizando que el entorno sea idéntico al de desarrollo.

5. Entornos en el Ciclo de Vida (SDLC)

Un flujo profesional requiere al menos tres entornos bien definidos:

- **Local / Development:** La máquina del desarrollador.
- **Staging / QA:** Entorno de pruebas que imita a producción para validaciones finales.
- **Production:** Donde los usuarios finales interactúan con la aplicación.

Posibles Preguntas de Examen

1. **¿Cuál es la diferencia técnica fundamental entre una Máquina Virtual y un Contenedor?**

- *Respuesta sugerida:* La Máquina Virtual incluye un Sistema Operativo completo y emula el hardware, lo que la hace pesada. El contenedor comparte el kernel del SO anfitrión y solo empaqueta la app y sus dependencias, siendo mucho más ligero y eficiente.

2. **¿Cómo ayuda Docker a cumplir con el principio de "Paridad de Entornos"?**

- *Respuesta sugerida:* Al crear una imagen inmutable que contiene exactamente las mismas versiones de librerías y configuraciones, se garantiza que el contenedor que corre en desarrollo sea idéntico al que corre en producción, eliminando fallos por diferencias de entorno.

3. **Explique los conceptos de Imagen y Contenedor en Docker.**

- *Respuesta sugerida:* Una imagen es el "molde" o archivo de solo lectura que define qué tendrá el entorno. El contenedor es el proceso en ejecución basado en esa imagen; es decir, la aplicación ya corriendo.

4. **¿Qué rol cumple Docker Hub en la Gestión de Configuración?**

- *Respuesta sugerida:* Actúa como el sistema de versionado para las imágenes de infraestructura, permitiendo almacenar, compartir y recuperar versiones específicas de los entornos de ejecución en la nube.
-